ucpqc: An Emulation-Based Fault-Analysis Platform for Post-Quantum Cryptography

Architecture, Detectors, Statistical Calibration, and Results

Vishwaajith N.K (23b1038)

October 17, 2026

Abstract

ucpqc is a Unicorn-based emulation platform that answers one mechanical question at scale: *if a fault is injected at this point in a post-quantum cryptographic implementation, does the output start leaking the secret key?* It boots real pqm4/pqcrystals Cortex-M4 firmware inside a full CPU emulator, injects a fault at a chosen site, and runs a *repertoire* of statistical detectors on the resulting outputs to decide LEAK/ok. The platform is deliberately scheme-agnostic: a thin *profile* layer supplies the algebra of each algorithm, while the emulator, fault injector, capture-replay engine, detector library, and assessment engine never mention a concrete scheme. The current build supports CRYSTALS-Dilithium (ML-DSA-44, lattice) and MAYO (multivariate), with the KEM path (ML-KEM/Kyber) present at the scheme layer. This report explains the whole codebase module by module, presents the recently added *statistically calibrated* verdict (per-site permutation / analytic p-values combined by a sweep-wide false-discovery-rate rule, replacing hand-tuned thresholds), and discusses the design tradeoffs and empirical results—including the headline Dilithium mask-removal leak, the honest limits observed on MAYO, and the emulation and parallelism cost/accuracy tradeoffs.

Contents

1 Introduction and Goals **2**

2 Background **3**
2.1 The two target schemes . . . 3
2.2 The fault model . . . 3

3 Architecture Overview **4**

4 The Emulation Core **4**
4.1 machine.py — the emulated Cortex-M4 . . . 4
4.2 platform.py — the board . . . 5
4.3 elfimage.py and tracing.py . . . 5

5 The Scheme Abstraction Layer **6**

6 Fault Models and Injection **6**

7 The Two Assessment Modes **6**
7.1 Mode `sweep` — whole-call skips . . . 7
7.2 Mode `funcskip` — intra-function skips via capture-replay . . . 7

8 Feature Extraction and the Detector Repertoire **7**
8.1 Why this signal: the matched filter versus exact recovery . . . 9

9 Statistical Calibration of the Verdict **10**
9.1 Per-site p-values . . . 10
9.2 Resolving below the empirical floor . . . 10
9.3 Sweep-wide false-discovery control . . . 11

10 Profiles and Extensibility **11**

11 Parallelisation **11**

12 Results **12**
12.1 Dilithium: the mask-removal leak (mode `sweep`) . . . 12
12.2 Auto-discovery: coverage versus cost . . . 13
12.3 Dilithium: intra-function skips (mode `funcskip`) . . . 14
12.4 LLVM-IR $\rightarrow$ ARM translation and key-dependent ineffectiveness (φ_{IF}) . . . 14
12.5 MAYO: an honest negative result . . . 15
12.6 Parallel speed and memory . . . 16
12.7 Correctness . . . 16
12.8 Positives and negatives, and the attacks they map to . . . 16

13 Design Tradeoffs **16**

14 Limitations and Future Work **18**

15 Conclusion **18**

A CLI Reference **18**

B Test and Verification Summary **19**

1 Introduction and Goals

Post-quantum cryptography (PQC) is being standardised (FIPS 203/204/205) and deployed on embedded targets, where an attacker has physical access and can inject *faults*—clock glitches, voltage glitches, laser pulses—to perturb a computation. Many PQC schemes have a structural weakness that a single well-placed fault can expose: a masking value that must stay secret, a rejection check that must not be bypassed, or an algebraic recombination that must not leak an intermediate. Evaluating a protected implementation against this class of attack by hand, one fault at a time, does not scale.

`ucpqc` is built to make that evaluation *systematic and generic*. Its design goals, in priority order, are:

1. **Solid detectors.** A LEAK verdict should be a *calibrated* statistical decision with a false-positive guarantee, not a magic threshold. The platform ships a repertoire of complementary detectors (mean-shift, per-coordinate, subspace, algebraic-structural, distributional, spec-boundary) unified under one calibration mechanism.
2. **Genericity across PQC families.** The same engine must assess lattice signatures, multivariate signatures, and (by construction) KEMs. Only a small per-scheme profile is written; everything else is shared.
3. **Catch many fault points.** Fault sites are *auto-discovered* by disassembly, and two fault surfaces are provided: whole-call skips and intra-function instruction skips.
4. **Test protected implementations for side effects.** Because the emulator is deterministic and reproducible, a hardened build can be assessed the same way as an unprotected one.

A deliberate scoping decision underlies the whole platform: **it decides leak / no-leak; it does not recover keys.** This keeps the analysis cheap (tens of traces, not millions) and the verdict interpretable, and it matches the practical question a countermeasure designer asks: "does this fault re-open a key-dependent channel?"

2 Background

2.1 The two target schemes

CRYSTALS-Dilithium / ML-DSA (lattice, Fiat–Shamir with aborts). Signing repeatedly samples a fresh masking nonce y, forms the response

$$z \;=\; y + c \cdot s_1 \pmod q,$$

where c is a sparse ± 1 challenge polynomial (expanded from the signature's $\tilde{c}$ seed) and s_1 is part of the secret key. The nonce y is a *mask*: it makes the marginal distribution of z independent of the secret, which is exactly why a plain histogram of z reveals nothing. The scheme then applies several *rejection checks* ($\|z\|$, $\|r_0\|$, $\|ct_0\|$, hint weight); if any fails, it resamples y and retries. Two fault-attack surfaces follow immediately: (i) remove the mask ($z = z + y \to z = c \cdot s_1$), and (ii) force a rejection check to accept, releasing out-of-spec signatures. ML-DSA-44 parameters used throughout: $q = 8380417$, $n = 256$ coefficients, $\ell = 4$, $\tau = 39$, $\gamma_1 = 2^{17}$, $\gamma_2 = (q-1)/88$, $\beta = 78$.

MAYO (multivariate, Oil-and-Vinegar). MAYO is *not* lattice-based. Its secret is a hidden "oil" subspace $O \subset \mathrm{GF}(16)^n$. Signing fixes fresh random *vinegar* variables (expanded from a seed by SHAKE), which makes the public map linear in the oil variables, then solves a linear system and outputs $s = v + o$ (vinegar + oil). The crown-jewel asymmetry is that *if an attacker learns or fixes the vinegar*, the signing equation becomes a solvable linear system that leaks the oil space—the key. Thus the fault-critical operations are the vinegar seed expansion, the linear solve, and the final recombination $s = v + o$. All arithmetic is over $\mathrm{GF}(16) = \mathrm{GF}(2^4)$.

2.2 The fault model

The physical faults `ucpqc` models are *instruction skip* (a fetch/execute glitch that turns an instruction, or a whole call, into a no-op), *data corruption* (bit-flip or stuck-at on a register or memory word), and *branch inversion* (flip a condition flag). These correspond one-to-one to the fault models in `faults.py` (§6).

3 Architecture Overview

`ucpqc` is layered so that scheme knowledge is confined to a single thin layer. Figure 1 shows the stack; Table 1 lists the modules.

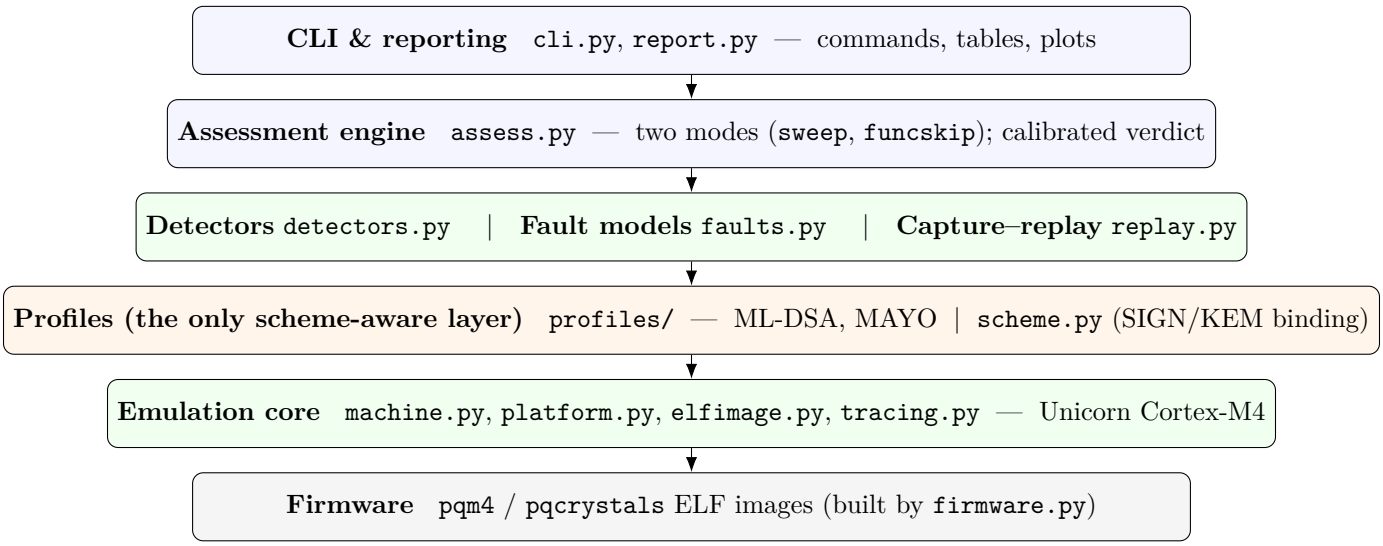


Figure 1: Layered architecture. Scheme-specific knowledge lives only in `profiles/` and `scheme.py`; every other layer works on instructions, memory, and feature matrices.

Table 1: Modules (≈5,300 lines of Python).

Module	Responsibility	LoC
`machine.py`	Cortex-M4 emulation, snapshot/restore, scratch, stub RNG, hooks	646
`tests/run_tests.py`	32-test system suite (runs on 3.12 and 3.14t)	523
`cli.py`	`python -m ucpqc` command front-end	470
`detectors.py`	Feature + detector + calibration library	432
`assess.py`	Scheme-agnostic assessment engine (two modes)	423
`replay.py`	Capture-and-replay for intra-function skips	396
`scheme.py`	SIGN/KEM detection and API binding	394
`faults.py`	Six fault models, injector, campaign runner	357
`tracing.py`	Instruction/call/memory tracers, profiler, `Trigger`	341
`firmware.py`	`pqm4` build wrapper + manifest	217
`platform.py`	MPS2-AN386 memory map + MMIO peripherals	201
`profiles/*`	`AnalysisProfile` + ML-DSA + MAYO	454
`elfimage.py`	ELF parsing / symbol lookup	163
`report.py`	Console table + bar-chart rendering	101

4 The Emulation Core

4.1 `machine.py` — the emulated Cortex-M4

`Machine` wraps a Unicorn `Uc` instance in Thumb/M-class mode running a Cortex-M4 model, plus a Capstone disassembler. It provides the primitives every higher layer relies on:

- **Boot and call.** `boot()` runs reset → C-runtime init → `main` (so `.bss` is zeroed and the FPU is on), then `call(func, args)` invokes a guest function with the AAPCS convention, returning through a mapped "trampoline" page. This is what lets the platform drive one function at a time rather than only running the whole firmware.
- **Snapshot/restore.** `snapshot()` captures registers (Unicorn context), all RAM regions, the instruction count, the scratch high-water mark, UART output, peripheral registers, *and* the position of the stubbed RNG stream. This makes a trial perfectly reproducible and is the mechanism behind per-site isolation (§7).
- **Deterministic RNG.** `stub_randombytes(seed)` replaces the guest's `randombytes` with a SHAKE256 stream (`_Xof`) that is a pure function of `(seed, position)`, hence rewindable. Determinism is essential: two keys must be compared under *identical* randomness so the only variable is the key.
- **Scratch allocator.** A framework-owned region (§4.2) supplies argument buffers via `alloc`, with `scratch_mark`/ `scratch_reset` to reclaim per-iteration buffers in a loop.
- **Hooks and interception.** `intercept(func, handler)` replaces a guest function with a Python callback (used for the RNG and the cycle counter); `hook_code`/`hook_mem` attach instruction/memory hooks. A subtlety handled here: Unicorn compiles hook checks into translation blocks, so adding a hook after boot requires flushing the block cache—otherwise it silently never fires on already-translated code.
- **Semihosting and cost.** ARM semihosting (`SYS_WRITE0`, `SYS_EXIT`, ...) is emulated for firmware output; a fast per-basic-block instruction counter (upgraded to per-instruction only when a precise index is needed) provides the cycle cost.

4.2 `platform.py` — the board

The default board is **MPS2-AN386**, the same target `pqm4` and QEMU use, so ELFs are byte-identical to what `qemu-system-arm -M mps2-an386` runs. Unicorn emulates only the CPU core, so peripherals are modelled as MMIO (Table 2): a CMSDK UART (captures firmware output), SysTick and the DWT cycle counter (both derived from the emulated instruction count), and a permissive fallback that stores unmodelled register writes rather than faulting.

Table 2: MPS2-AN386 memory map (`platform.py`).

Region	Base	Size	Role
`flash`	`0x00000000`	4 MiB	code+rodata+data+bss (`DATA_IN_FLASH=1`)
`sram`	`0x20000000`	4 MiB	stack (grows down from top)
`scratch`	`0x30000000`	1 MiB	framework-owned call arguments
`trampoline`	`0x1F000000`	4 KiB	return address for direct calls
`apb`	`0x40000000`	1 MiB	CMSDK UART/GPIO/timers (MMIO)
`ppb`	`0xE0000000`	1 MiB	SysTick, SCB, DWT (MMIO)

4.3 `elfimage.py` and `tracing.py`

`elfimage.py` parses the ELF, indexes function symbols (with synthesised extents for size-less symbols), and offers `addr_of`, `func_at`, `extent_of`, and `describe(addr)`→`"func+offset"`. `tracing.py`

provides a `Trigger` (entry/return boundary detection used by capture-replay) and instruction/call/memory tracers plus a per-function `Profiler`, all attachable to a live machine and scopable to one function.

5 The Scheme Abstraction Layer

`scheme.py` is the *only* scheme-aware layer below the profiles. From the ELF symbol table alone it detects whether the firmware is a signature or a KEM (pattern-matching `crypto_sign_*` vs `crypto_kem_*`, including PQClean and pq-crystals namespacing), binds the entry points, and resolves buffer sizes from a build manifest, a built-in size table, or by probing (running once with fill patterns). It then exposes a uniform API—`keypair`, `sign`, `verify` for signatures; `encaps`, `decaps` for KEMs; and a generic `roundtrip`. Each method is exactly one `Machine.call`, so any fault or hook installed on the machine applies to precisely that operation.

The size table already knows ML-DSA-44/65/87, Dilithium2/3/5, ML-KEM/Kyber 512/768/1024, Falcon-512/1024, MAYO-1, and SPHINCS+-128f/s. The KEM path (`encaps`/`decaps`) is fully implemented and tested at round-trip; what is missing for KEM *assessment* is only a profile (§14).

6 Fault Models and Injection

`faults.py` defines six fault models (Table 3) as a `FaultSpec` (*what* to corrupt) plus a trigger (*when*): either a global instruction index `at` (hooks every instruction) or a program point `pc` firing on its `hit`-th execution (hooks one address—much cheaper, so a whole-function sweep costs about as much as the function itself). An `Injector` installs one spec and reports whether it fired.

Table 3: Fault models (`faults.py`). All six exist in the injector; the systematic detector sweep currently drives `SKIP` (see §14).

Model	Effect	Physical analogue
`SKIP`	step PC past k instructions	clock/laser glitch on fetch
`BITFLIP_REG`	flip one register bit	transient on the register file
`SET_REG`	force a register to a value	stuck-at fault
`BITFLIP_MEM`	flip one memory bit	transient in RAM/bus
`SET_MEM`	force a memory word	data corruption / zeroization
`FLIP_FLAGS`	invert a CPSR condition flag	branch inversion

A `FaultCampaign` keeps one booted machine, snapshots before a golden run, and for each trial restores the snapshot, injects one fault, and re-runs an operation—classifying the outcome by comparison with the golden output (SILENT, DIFFERENT, CRASH, TIMEOUT, REJECTED, SKIPPED). This campaign path (exposed as the `fault` command) answers "did the fault change the output / break the operation?". It is *distinct* from the leak-detector sweep of §7, which asks the sharper question "did the fault make the output leak the key?". Unifying the rich fault models here with the detector repertoire there is the single largest planned extension (§14).

7 The Two Assessment Modes

The assessment engine (`assess.py`) has two modes that share one spine: two keys A, B; N messages signed under each with identical nonce streams (so the key is the only variable); a fault at one site;

a per-output *feature*; a *detector*; and a LEAK/ok/crash verdict.

7.1 Mode `sweep` — whole-call skips

For each `bl`/`blx` call site in the signing loop (auto-discovered by disassembling the resolved signing function, descending through thin wrappers), the engine installs a persistent skip of that whole call and re-signs. Fault sites are therefore *firmware-agnostic*: no hand-authored address table.

Per-site isolation. A faulted or hung signing leaves the guest machine dirty. Without isolation the next site would inherit that state and its verdict would depend on execution order—reproducible serially but non-deterministic once a pool schedules sites across workers. `_run_site` therefore snapshots on entry and restores on exit, so every site starts from identical clean state. This one change (i) made serial and both parallel backends agree exactly and (ii) removed the spurious near-threshold "leaks" that small-N sweeps used to show.

7.2 Mode `funcskip` — intra-function skips via capture-replay

A full ML-DSA signing is $\sim$1.5 M+ instructions (with retries), so sweeping hundreds of instruction-skip sites $\times N$ messages $\times$ 2 keys by re-signing is prohibitive. `replay.py` instead *captures* the target function's I/O once per golden signing, then *replays* only that function (a few thousand instructions) under a skip at each interior site—roughly **240× cheaper** than re-signing. Two backends (Table 4): `snapshot` restores full machine state and resumes entry→return (works for any region); `call` rebuilds the call from captured arguments on a fresh stack (cheap storage, functions only, and it carefully *preserves argument aliasing* so an in-place op like $z = z + y$ replays correctly).

Table 4: Capture-replay backends (`replay.py`).

Backend	Mechanism	Applies to	Stored per capture
`snapshot`	restore state, resume entry→ret	any region/function	full RAM snapshot
`call`	rebuild call on fresh stack	functions only	args only (small)

8 Feature Extraction and the Detector Repertoire

A detector consumes per-item feature vectors, shape (N, D), with the two key populations passed as separate arrays. The right detector depends on the *role* of the faulted quantity. Table 5 summarises the repertoire; the key idea is that `two_key`, `per_coord`, and `subspace` are the *same* leave-one-out classifier under identity, diagonal, and full covariance respectively, and `structural` is that same idea over the scheme's own finite field.

The matched-filter feature. Dilithium's z has a secret-independent marginal, so a histogram is blind; the leak lives in the *joint* (c, z) relation. `matched_filter` extracts it as the negacyclic correlation of the sparse challenge c against each response polynomial, yielding an $\ell{\cdot}n$ feature that a leaky site separates cleanly.

Table 5: The detector catalog (`detectors.py`, mirroring `examples/DETECTIONS.md`): the feature each detector consumes, its statistic and legacy fixed cutoff, how the calibrated verdict is obtained, and what it catches. `two_key` is the default detector; `per_coord` (TVLA) is now the sweep default.

Detector	Feature consumed	Statistic · legacy cutoff	Calibrated by	Catches
`two_key`	matched filter of c against z (or raw bytes)	LOO nearest-mean accuracy · $\geq 80\%$	permutation p	whole-output **mean shift** by key
`per_coord`	same feature	$\max \lvert t \rvert$ (Welch) over coords · $\geq$ Bonferroni ($\sim$5.3)	analytic (Bonferroni normal tail)	leak in **one output position**
`subspace`	same feature	covariance-whitened (LDA) LOO acc. · $\geq 80\%$	permutation p	leak in a **correlated subspace**
`structural`	**field elements** ($\mathbb{Z}_q$ / GF(16))	rank-collapse score (collapse · sep.) · ≥ 0.5	permutation p (sharp; null ≈ 0)	**accumulation / recoverability**
`mmd`	same feature	kernel two-sample MMD^2	permutation p	any **distributional** shift by key
`differential`	raw z + **golden** baseline	$\lvert t \rvert$ of per-item change magnitude between keys · $\gtrsim 2.6$	analytic (normal tail)	a **key-dependent fault effect**
`uniformity`	raw nonce coefficients	largest histogram bin share · $> 10\%$	analytic (χ^2 tail)	a **biased nonce** (loop abort)
`spec_aware`	raw coeffs + `reject_bound`	count of coeffs $\geq$ bound · > 0	deterministic (p=0 if >0)	a **rejection-boundary bypass**
`r0_reject`	captured accepted `r0` + bound	share of sigs. with out-of-spec `r0` · > 0	deterministic	**r0 rejection** / Finding-a-Polytope

The two-key classifier. For per-key mean templates g_A, g_B, each item gets a leave-one-out discriminant score

$$\text{score}(x) \;=\; \langle x, g_{\text{own}}^{\text{LOO}} \rangle \;-\; \langle x, g_{\text{other}} \rangle,$$

leaving x out of its own template so it never helps classify itself (essential at $D = 1024$, $N \approx 50$, or in-sample noise would look separated). Accuracy 0.5 means indistinguishable; 1.0 means fully separable. `subspace` first whitens by the pooled within-key covariance (shrunk toward a scaled identity, with a PCA reduction to bound its rank at small N), so it also sees leaks confined to a low-dimensional correlated subspace.

The field-aware structural detector. This is the algebra-aware generalisation. Each key's rows are integer matrices of *field elements* (for Dilithium, per-signature s_1 estimates recovered by deconvolving c from z over $\mathbb{Z}_q[x]/(x^n + 1)$; for MAYO, exposed oil bytes over GF(16)). A leak makes each key's rows collapse to a low-dimensional, key-specific subspace over the field. With per-key ambient rank full $= \min(\min(N_A, N_B), D)$,

$$\text{collapse} = 1 - \frac{\max(r_A, r_B)}{\text{full}}, \quad \text{separation} = \frac{r_{AB} - \max(r_A, r_B)}{\max(\min(r_A, r_B), 1)}, \quad \text{score} = \text{collapse} \cdot \text{separation}.$$

Rank is computed over a pluggable `Field` backend: `PrimeField(q)` (numpy Gaussian elimination mod p with Fermat inverses) for $\mathbb{Z}_q$, and `GF2m(m,poly)` (log/antilog tables, add = XOR) for GF(2^m). Golden/masked outputs stay full rank, so the score is ≈ 0; a genuine collapse pushes it

toward 1. The finite-field ranks are cross-checked against the `galois` library in a separate 3.12-only oracle test.

8.1 Why this signal: the matched filter versus exact recovery

Every `sweep` detector reads a feature derived from the released response z, so the first design question is *which* function of the output to look at. Dilithium publishes $z = y + c\,s_1$ per response polynomial, where the challenge c is *known* (recomputed from $\tilde{c}$ via the firmware's own SampleInBall) and sparse ternary ($\tau = 39$ coefficients ± 1, the rest 0), and y is a fresh uniform mask. Rejection sampling makes the *marginal* of z secret-independent — a histogram of z alone reveals nothing — so a leak can only surface in the *joint* (c, z) relation: conditioned on the known c, the mean of z is $c\,s_1$, which depends on the secret. Any usable feature is therefore a function of (c, z) that exposes this conditional term, and that term is exactly what the fault- and side-channel literature on Dilithium attacks.

Two ways to expose it. Multiplication by c in $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ is linear in s_1; write it as the negacyclic (anti-circulant, with the $x^n = -1$ sign flips) matrix C, so that $z = C\,s_1 + y$. The secret term can be read off in two canonical ways:

- **Exact recovery** $\hat{s}_1 = C^{-1}z$ (ring deconvolution): with the mask removed this returns s_1 exactly. It is literally the recovery step of the loop-abort / nonce-zeroing fault attacks[1] and the "Finding a Polytope" fault attack[2], and it is the target variable of DPA/CPA on the $c\,s_1$ multiply[3].
- **Matched filter** $C^\top z$ (correlation of c against z): this is the right-hand side of the normal equations $C^\top C\,s_1 = C^\top z$ for that same system. Because c is sparse ternary, $C^\top C \approx \tau I$ (its autocorrelation peaks at $\tau = 39$, with small sidelobes), so $C^\top z \approx \tau\,s_1$ once the mask is gone — a cheap, well-conditioned approximation of the exact inverse. `matched_filter` computes exactly this, summing only over c's τ nonzero taps, so it is $O(\tau n)$ and exact (not truncated).

Why $C^\top z$ for detection. Both expose the same conditional term, but they behave very differently as a *bulk* classifier feature at $D = 1024$, $N \approx 50$:

- *Conditioning.* $C^{-1}z = s_1 + C^{-1}y$ spans all of $\mathbb{Z}_q$ ($\sim 2^{23}$); the key-bearing s_1 (coefficients in $\{-2, \ldots, 2\}$) is buried under a near-uniform term, and the occasional non-invertible challenge collapses to a zero vector. A leave-one-out nearest-mean rule over so few high-dimensional samples then latches onto structure that is not the leak. Empirically, on the `ml-dsa-44` sweep ($N = 24$), feeding the exact recovery $C^{-1}z$ into the same two-key classifier reads accuracy 0.92 ($p = 0.018$) at the *unfaulted control* — a false positive — whereas the matched filter stays at 0.17. Both agree at 1.00 ($p = 0.002$) on the genuine mask-removal site (`polyvecl_add`).
- *Robustness and cost.* $C^\top z$ is bounded, needs no ring inverse, is pure-numpy $O(\tau n)$ ($\sim$30$\times$ faster than the sympy deconvolution), and has a clean null — the properties a first-pass yes/no sweep across tens of sites needs.
- *Detection vs. recovery.* The exact inverse is what an *attacker* runs once a site is flagged, and it stays available in the tool as the `structural` detector, which feeds $C^{-1}z$ into a finite-

[1] T. Espitau *et al.*, "Loop-abort faults on lattice-based Fiat–Shamir and hash-and-sign signatures," SAC 2016; L. Groot Bruinderink and P. Pessl, "Differential Fault Attacks on Deterministic Lattice Signatures," TCHES 2018(3); see also Ulitzsch *et al.*, "Defeating Fault Countermeasures in Lattice Signatures," TCHES.

[2] Azevedo-Oliveira *et al.*, "Finding a Polytope: a practical fault attack against Dilithium," ePrint 2025/195 — the observable behind the `r0_reject` detector.

[3] Ravi *et al.*'s DPA on the pointwise $c\,s_1$; a non-profiled CPA variant is ePrint 2024/111.

field *rank* test (Section 8): its statistic is rank rather than distance, so it is immune to the dynamic-range problem above. For the first-pass detection sweep we deliberately use the best-conditioned projection of the signal, the matched filter.

Provenance. The "correlation = matched filter" framing (treating the sparse ± 1 challenge as a spreading code whose autocorrelation despreads $c\,s_1$) is standard signal processing, but we did not find it named as such in the Dilithium literature. We therefore present `matched_filter` not as a cited primitive but as a well-conditioned reformulation of the recovery / CPA quantity ($C^{-1}z$ / $c\,s_1$) that the works above target.

9 Statistical Calibration of the Verdict

The original engine decided LEAK by comparing a raw statistic to a hand-picked constant (accuracy ≥ 0.80, structural score ≥ 0.50, bin share > 0.10). These cutoffs have no false-positive meaning, do not adapt to N or D, and—applied independently across 30–150 swept sites—have no multiple-testing control, which produced spurious near-threshold "leaks" at small N. The most recent work (branch `calibrated-detectors`) replaces them with a calibrated verdict shared across the whole repertoire, while keeping a `-legacy-thresholds` flag that restores the old cutoffs exactly.

9.1 Per-site p-values

Each detector's statistic becomes a per-site p-value, chosen appropriately for the detector:

- **Classifier / structural** (`two_key`, `subspace`, `structural`, `mmd`): a *permutation* test under the null that the two key populations are exchangeable. Shuffling the key labels n_{perm} times builds the null distribution of the statistic T, giving the assumption-free empirical p-value

$$p_{\text{emp}} = \frac{1 + \#\{T_b \geq T_{\text{obs}}\}}{n_{\text{perm}} + 1},$$

  floored at $1/(n_{\text{perm}} + 1)$. One generalised routine, `perm_pvalue(stat_fn, A, B)`, serves every classifier by taking the statistic as an argument.

- **Per-coordinate** (`per_coord`): the exact analytic dual of the TVLA threshold, $p = \min(1,\ d \cdot 2\,\Phi^c(\max|t|))$ (Bonferroni over the d within-site coordinates).

- **Uniformity**: the analytic χ^2 tail $p = \chi^2_{\text{sf}}(\cdot, \text{dof})$.

- **Spec-aware**: deterministic—any out-of-bound coefficient is a real violation, so $p = 0$ when present.

9.2 Resolving below the empirical floor

When the observed statistic beats *every* permutation, p_{emp} is pinned at the floor. To recover resolution, `perm_pvalue` also returns a *studentised Gaussian tail* $p_{\text{tail}} = \Phi^c\big((T_{\text{obs}} - \mu_{\text{null}})/\sigma_{\text{null}}\big)$, and `_resolve_p` uses it *only* at the floor:

$$p \;=\; \begin{cases} p_{\text{emp}}, & p_{\text{emp}} > \frac{1}{n_{\text{perm}}+1}, \\ \min(p_{\text{emp}},\, p_{\text{tail}}), & \text{otherwise.} \end{cases}$$

This is honest about a real property: the tail is *sharp* when the null is concentrated (the structural rank score has a degenerate null at ≈ 0, so $\sigma_{\text{null}} \to 0$ and $p_{\text{tail}} \to 0$) and *weak* when the null is wide (`two_key` at small N). Consequently an isolated structural leak clears any sweep-wide threshold regardless of site count, whereas an isolated classifier leak in a very large sweep needs n_{perm} raised until the floor clears q/m (documented; the default $n_{\text{perm}} = 1000$ suffices for the tens of sites a `funcskip` target has).

9.3 Sweep-wide false-discovery control

A sweep tests many sites, so the engine collects all per-site p-values and applies a multiple-testing rule in a second pass (`_apply_correction`). The default is Benjamini–Hochberg: with sorted $p_{(1)} \le \cdots \le p_{(m)}$, reject all up to the largest k with $p_{(k)} \le \frac{k}{m} q$, controlling the false-discovery rate at q (default 0.01). `-correction holm` switches to Holm–Bonferroni for family-wise error control. The verdict now reads "these sites leak at FDR $\le q$ across the whole sweep," with *no per-scheme tuning*.

Crucially, this two-pass structure keeps the per-site worker (`_run_site`) *pure*: it emits `(metric, p)` and the FDR reduction runs once where rows are collected. That is what lets the serial engine and both parallel backends produce identical verdicts, and lets the parallel branches inherit calibration on a clean rebase.

10 Profiles and Extensibility

A new scheme is one module implementing `AnalysisProfile`. The engine calls only this interface, so no engine change is needed. Table 6 lists what a profile supplies, by detector.

Table 6: What a new algorithm must supply (`profiles/__init__.py`).

For detector(s)	The profile adds
`two_key/per_coord/schoolbook`	`challenge`, `response_from_*`, `feature` (a real-valued, leak-aligned feature, e.g. the matched filter)
`uniformity/spec_aware`	the raw-coefficient response and the relevant bound (`reject_bound`)
`structural`	`field` (a `PrimeField`/`GF2m`) *and* `structural_feature` returning a row of field elements
`funcskip` mode	`targets()`/`default_target` (a `replay.Target`)

Fault-site discovery, the mode engine, the detectors, and the entire calibration are scheme-agnostic; a new scheme supplies *no thresholds*. Two profiles ship: **`MLDSAProfile`** (matched filter; `PrimeField(q)` with an s_1 deconvolution) and **`MayoProfile`** (raw released bytes; `GF2m(4)` with oil nibbles). A notable cleanliness decision: the Dilithium $\mathbb{Z}_q[x]/(x^n + 1)$ challenge inverse is delegated to `sympy` (a well-tested, *pure-Python* library) rather than a hand-rolled NTT—which had already produced two bugs (an int64 overflow and a rank-normalisation error). Pure-Python matters because the free-threaded build must not re-enable the GIL (§11).

11 Parallelisation

Sites are independent once isolated, so the sweep parallelises well. Because Unicorn `Machine`s and `Snapshot`s hold C handles (non-picklable), the design is kept on two branches that share the pure

per-site seam on `master`:

- **`parallel-multiprocessing`**: `ProcessPoolExecutor` with the `spawn` start method; each worker builds its own machine from the deterministic ELF path—no shared C state.
- **`parallel-freethreading`**: `ThreadPoolExecutor` on free-threaded CPython 3.14t, with a per-thread machine held in `threading.local`.

The measured tradeoff is memory versus isolation: at `-j20`, threads use ≈156 MB against ≈1.7 GB for processes (∼11×), since processes duplicate the interpreter and firmware image per worker. Free-threading is only viable because every hot dependency is pure-Python or releases no GIL constraint: `numpy`/`scipy`/`sympy` are fine, whereas `numba` (pulled in by `galois`) has no free-threaded build—so `galois` is a *test-only* dependency confined to the 3.12 oracle tests and never reaches the 3.14t environment.

12 Results

Table 7 is the one-line detection log — every leak the framework **flagged** (and the notable ones it deliberately does not), with the test inputs and the detection method for each. The subsections that follow expand each row; the detection log itself is maintained in `examples/DETECTIONS.md`.

Table 7: Detection log: what was flagged, on what inputs, by which detector (`examples/DETECTIONS.md`).

Scheme	Site / fault	Inputs	Detector	Metric	Verdict
ML-DSA	`z=z+y` skip (`bl polyvecl_add`)	2 keys × N, matched filter(c, z)	`two_key`	100%	**LEAK**
ML-DSA	same, field-aware	2 keys × N, s_1 over $\mathbb{Z}_q$	`structural`	0.917	**LEAK**
ML-DSA	`mov r7,r0` (out ptr)	funcskip isolated replay	`two_key`	100% iso / 69% e2e	artifact
ML-DSA	r0-check forced accept	raw coeffs + bound	`spec_aware`	`band_count`>0	**LEAK** (ex. 08)
ML-DSA	`poly_chknorm` skip	—	`two_key`	—	crash/hang
MAYO	any sweep site	2 keys × N, raw bytes	`two_key`	≤54%	none (control 46%)
MAYO	de-inlined `mat_add` eor-skip	funcskip, GF(16)	all	≤57% / 0.0	none (needs data fault)

12.1 Dilithium: the mask-removal leak (mode `sweep`)

Across the auto-discovered signing-loop sites, exactly one leaks: the masking add $z = z + y$ (`bl polyvecl_add`, `0x4d5e`), at **100%** two-key accuracy; skipping it leaves $z = c \cdot s_1$, exposing the secret. Every other site either sits at chance or hangs (Figure 2). This is a documented fault (the "Exploiting Determinism" family). Re-expressed as TVLA, the same site shows $\max|t| = 15.4$ with 444 of 1024 coordinates past the 4.5 threshold, against a ≈ 3.5 noise ceiling everywhere else; a 2000-shuffle permutation test gives $p = 0.0005$ (the floor) at the leak versus $p = 0.46$ for the control (Figure 3).

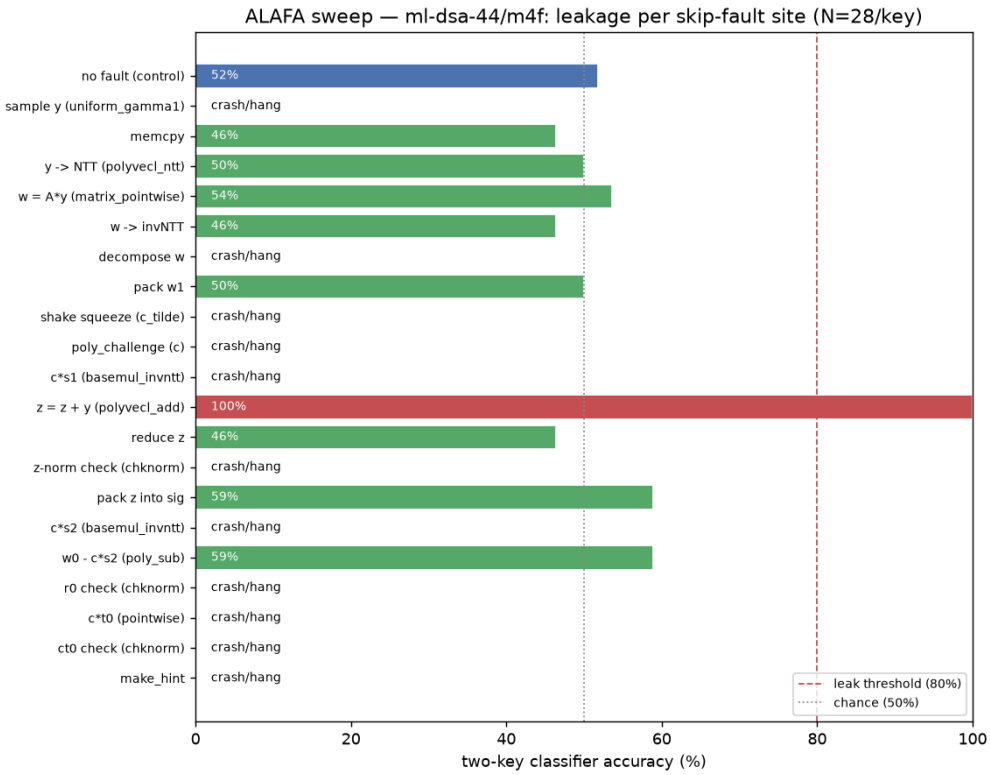


Figure 2: ALAFA sweep verdict: only $z = z + y$ crosses; other sites crash/hang or sit at chance.

Why most sites hang. A whole-call skip usually drops a value some rejection check depends on, so the reject-and-retry loop *never accepts* and spins to the 20 M-instruction cap. This is a real fault-model observation: *skipping a check and forcing its verdict are opposite faults.* Skipping `bl poly_chknorm` leaves a stale nonzero result → perpetual reject (hang); the check-bypass *leak* needs a stuck-at-accept data fault instead. So the whole-call sweep is honestly scoped to high-SNR mask-removal faults.

12.2 Auto-discovery: coverage versus cost

Replacing the hand-authored 20-site table with disassembly-based discovery is a strict *superset*: all 20 curated sites appear among 48 discovered, and the one leaking site is still flagged. The cost is a ∼5.7× slowdown (Table 8)—worse than the 48/20 site ratio predicts, because most of the 28 extra sites are load-bearing setup calls whose skip makes signing hang to the cap. The trade accepted: fully automatic, firmware-agnostic site generation in exchange for more grey (hang) rows.

Table 8: ALAFA sweep: coverage/cost of auto-discovery (ML-DSA-44, N=12/key).

Sweep	Sites	Leaks found	Wall time
Curated table	20	1 (z=z+y)	52.3 s
Auto-discovered	48	1 (z=z+y)	299.6 s (∼5.7×)

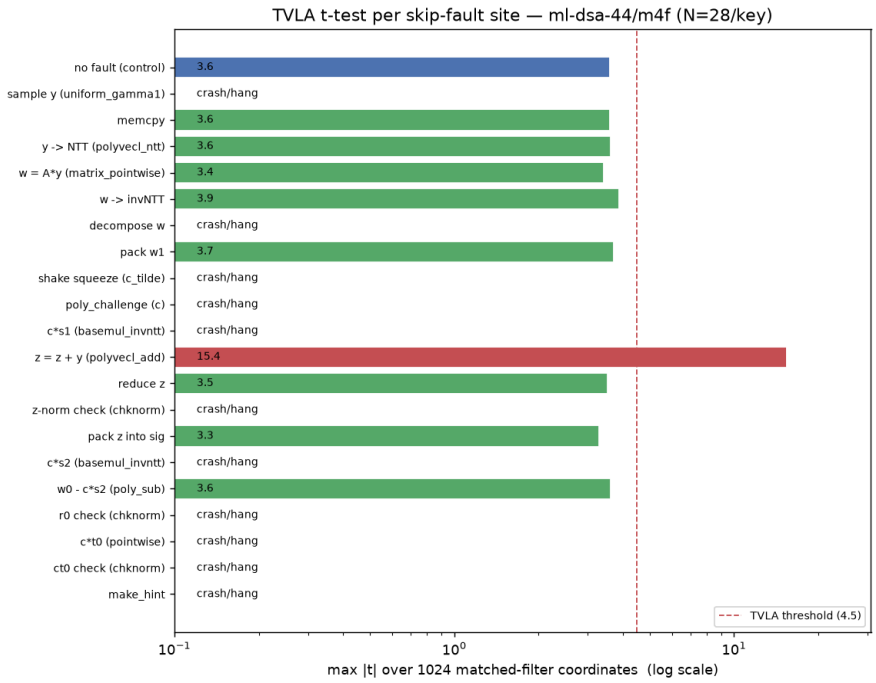


Figure 3: TVLA twin of the verdict: only the mask-removal site rises above the threshold.

12.3 Dilithium: intra-function skips (mode `funcskip`)

Inside `polyvecl_add`, the calibrated `two_key` detector flags exactly two sites—the output-pointer load `0x46d2` and the mask add `0x46e4`—at the permutation floor $p \approx 10^{-3}$, FDR ≤ 0.01, with non-leaking sites at $p \in [0.57, 0.96]$. The `structural` detector flags the *same* two sites with $p = 0$ (its degenerate-null tail) and *no* false positives, every other site sitting at $p = 1.0$. Listing 1 shows the actual calibrated output; the `-legacy-thresholds` run reproduces the same two sites via the old 0.80 cutoff.

Listing 1: Calibrated `funcskip` on ML-DSA-44 (`two_key`). The p column and the FDR-based footer are the calibrated verdict.

```
  addr  site                          metric      p  ran crash  status
0x46d2  mov r7, r0                       100%  0.0010  16   0    LEAK  <- LEAK
0x46d4  mov r6, r1                        50%  0.5684  16   0    ok
0x46d6  mov r5, r2                        31%  0.8981  16   0    ok
0x46e4  bl #0x3d94                       100%  0.0010  16   0    LEAK  <- LEAK
0x46e8  cmp.w r4, #0x1000                 25%  0.9600  16   0    ok
→ 2 leaking site(s) at FDR≤0.01: 0x46d2 mov r7, r0, 0x46e4 bl #0x3d94
```

12.4 LLVM-IR → ARM translation and key-dependent ineffectiveness (φ_{IF})

Mapping the DILITHIUM-LLVM IR-level fault findings onto the compiled ARM binary—`funcskip` over the 29 reference functions, n=40 signatures per key—sharpens two detector axes that must not be conflated: the *correction* axis (the faulted *output distribution* differs by key: our `per_coord`/TVLA test) and the *ineffective* axis (whether the skip is a *no-op* differs by key). On the correction axis only `polyvecl_add` leaks (5/10 sites, max $|t| \approx 16.9$ against a 3–4.5 no-leak floor), so IR value-corruption findings translate poorly to whole-instruction skips; and the `m4f` compiler inlines 8 reference functions (`polyveck_add/sub/chknorm/make_hint`, the $c \cdot s$ `*_pointwise_poly_montgomery`

multiplies, `polyvecl_invntt/pointwise_acc`) out of the signing path, shrinking the fault surface below the IR the tool analysed.

The ineffective axis is the SIFA-style test φ_{IF}: a *key-dependent* ineffective fault, $\exists\, p\ \exists\, s_1, s_2$: $\Delta(\mathrm{fault}(s_1, p), \mathrm{gold}(s_1, p))=0 \ \wedge\ \Delta(\mathrm{fault}(s_2, p), \mathrm{gold}(s_2, p))\neq 0$. We test it per skip site with keys A, B signing the *same* message and signing-RNG (only `sk` differs), counting *flips*—messages where the skip is a no-op for one key but output-changing for the other. This surfaces 71 φ_{IF} sites across four reachable functions (Table 9), where the earlier rate-based `sifa` detector—comparing no-op *rates* across unrelated signings—had flagged none.

Table 9: Key-dependent-ineffective (φ_{IF}) sites on the ARM skip surface, and the per-signing input whose value drives the no-op/effective split at each. (Whether a value is secret- or public-derived does *not* by itself decide SIFA-exploitability; see text.)

Function	φ_{IF} sites (flips)	Input driving the split
`ntt`	63 (261)	NTT of masked/secret polynomials (secret-derived)
`poly_challenge`	5 (7)	challenge seed $\tilde{c} = H(\mu \,\|\, w_1)$ (in σ)
`polyvecl_uniform_gamma1`	2 (38)	mask y from $\rho' = H(K \,\|\, \mu)$
`poly_chknorm`	1 (1)	norm check of the response z

Two observations qualify these counts. First, they are *partial*: most sites flip on only a handful of the 40 messages, because a single-instruction skip is a no-op only under a rare boundary condition—a redundant store, a dead write later overwritten, an unused sign bit. At `poly_challenge`'s coefficient store `0x4122` the skip is *effective* in 38/40 messages for A and 39/40 for B; a flip needs one key to hit the rare no-op where the other does not, which occurs at just 3 messages. Second, the metric must be read with care. A φ_{IF} flip only says the two keys' runs *differed* in ineffectiveness at some message; but A and B each perform a full signing and therefore also feed *different inputs* to the faulted function (different $\tilde{c}$, different μ via $\mathrm{tr} = H(pk)$), so a flip conflates genuine key-dependence with ordinary input-dependence. φ_{IF} thus *locates candidate sites*; it does not, on its own, establish SIFA-exploitability. Crucially, exploitability is *not* decided by whether the driving value is public: SIFA conditions on the ineffective faults and exploits a resulting *bias* in an intermediate that correlates with the secret, and that bias can arise even when the faulted operand is entirely public—faulting the Fiat–Shamir challenge generation and filtering on valid signatures is itself a studied vector. Settling each site therefore needs the SIFA test proper: fix the secret, inject the fault across many inputs, filter to the ineffective subset, and test whether a target intermediate's *conditional* distribution is biased toward secret-correlated states (§14). What the present study does establish is a *correction* to the earlier rate-based reading: key-dependent ineffectiveness is *present* on the skip surface rather than absent, and it is heaviest in the NTT butterflies (`smull`/`smlal`/`mul`/`add`), where the `m4f` compiler relocated the secret arithmetic after inlining the $c \cdot s$ multiplies away.

12.5 MAYO: an honest negative result

On MAYO the whole-call sweep finds *zero* leaking sites, and this is the correct answer for the current fault surface. The control sits at chance (46%): fresh vinegar masks the key just as y does in Dilithium. Every vinegar-critical site (SHAKE seed expansion, the linear solve, `compute_M_and_VPV`) *crashes* under a whole-call skip, because zeroing the whole vinegar/seed makes the linear solve fail; and the recombination $s = v + o$ is *inlined*, so there is no `bl` to skip. Even after rebuilding MAYO with the recombination de-inlined and capturing it with `funcskip`, an instruction skip yields chance across all detectors: the fused, word-vectorised GF(16) loop exposes the *vinegar* operand (SHAKE-random, not separable), and the oil leak is GF(16)-*algebraic*. What would flag it is a *data fault*

(zero/fix an operand) that leaves clean oil, then the GF(16) `structural` detector—the detector and localiser are in place; the missing piece is wiring the data-fault model into the sweep. This is a precise, useful negative result: it says exactly which capability the platform still needs.

12.6 Parallel speed and memory

The MAYO sweep (heavily hang-dominated) runs in 566.8 s serially and 37.4 s at `-j20`. As noted (§11), threads cost ≈ 156 MB against ≈ 1.7 GB for processes at the same width—the platform's central parallelism tradeoff.

12.7 Correctness

The 32-test system suite passes on both standard CPython 3.12 and free-threaded 3.14t (with the GIL verified *off*), and the `galois` oracle confirms the hand-rolled $\mathbb{Z}_q$ and GF(16) ranks against a trusted library. The calibration change preserved every known verdict (the two `funcskip` sites, the ALAFA leak) while removing the small-N false positives.

12.8 Positives and negatives, and the attacks they map to

Table 10 classifies the findings as *positives* (a leak flagged) or *negatives* (nothing flagged), and names the published attack each positive corresponds to — so a detection reads as evidence for a concrete, documented fault rather than an abstract score. The negatives are kept deliberately: a detector framework earns trust from its honest misses as much as its hits. References are the footnotes in Sections 8–8.1.

13 Design Tradeoffs

Emulation vs. hardware.
Emulation gives perfect determinism, full observability, byte-identical firmware, and cheap snapshot/restore—at the cost of not modelling analogue effects (partial glitches, timing, EM). `ucpqc` embraces this: it is a *fault-effect* analyser ("does this logical fault re-open a channel?"), not a physical side-channel simulator. It cleanly models the digital consequence of a glitch (skip / bit-flip / stuck-at) and is explicit that a *partial* nonce abort (Dilithium category 4.1) is outside the whole-call model.

Leak-detection vs. key recovery.
Deciding leak/no-leak needs tens of traces and yields an interpretable verdict; full key recovery needs many signatures and heavier cryptanalysis. The platform picks the former deliberately, which is what makes an exhaustive per-site sweep affordable.

Whole-call `sweep` vs. `funcskip`.
`sweep` is coarse but needs no target description and covers the whole loop; `funcskip` is fine-grained (every instruction) and $\sim$240$\times$ cheaper per trial, but needs a `Target` (arg layout) and can surface isolated-replay artefacts (e.g. the `0x46d2` pointer-aliasing site) that a full signing would overwrite.

Auto-discovery vs. curated sites.
Disassembly-based sites are zero-config and a strict coverage superset, at $\sim$5.7$\times$ runtime because most extra sites hang to the instruction cap. Coverage and portability were judged worth the cost.

Calibrated p-values vs. fixed thresholds.
Calibration gives every verdict a false-discovery guarantee and removes per-scheme tuning; the cost

Table 10: Positives (a real leak flagged, with the corresponding published attack) and negatives (nothing flagged, with the reason).

Observation (site / fault)	Detector · result	Corresponding attack / reason
Positives — a real leak, flagged		
Mask removal $z = c\,s_1$ (skip `polyvecl_add`)	`two_key` 100%, `structural` 0.92, `per_coord` $\lvert t\rvert$=12.9	"Exploiting Determinism" (Ravi *et al.*); loop-abort (Espitau *et al.*, SAC'16); Diff. Fault (Bruinderink–Pessl, CHES'18)
Out-of-spec accepted r_0 (forced accept, ex. 08)	`spec_aware` / `r0_reject`	"Finding a Polytope" (PKC'25); the r_0-rejection bypass
Biased nonce y (loop abort, `funcskip`)	`uniformity`	loop-abort / nonce-zeroing (Espitau *et al.*)
Key-dependent ineffectiveness (planted)	`sifa`	SIFA (Dobraunig *et al.*, 2018)
Key-dependent fault *effect* (planted)	`differential`	differential fault analysis (general)
Negatives — nothing flagged (honest), and why		
MAYO skip sweep (any site)	all $\leq 54\%$ / 0.0 $\rightarrow$ none	oil leak is GF(16)-algebraic; needs a *data* fault exposing clean oil, then `structural` over GF(16)
`sifa` / `r0_reject` on the skip surface	0 flagged	need the data-fault surface (`SET_MEM`/`BITFLIP`); skip-only hangs or yields no r_0
`poly_chknorm` skip	crash/hang	stale verdict $\rightarrow$ infinite reject loop (not a leak)
`per_coord`/TVLA at the unfaulted control	$p \approx 0.007$ (false positive)	small-N normal tail is anti-conservative; tolerated for a detector framework
`differential` / `sifa` across the skip sweep	0 new after FDR	the skip surface's only *correction* leak is the mask removal (`two_key`'s); the direct φ_{IF} reading (§12.4) finds key-dependent ineffectiveness *present* (heaviest in the NTT), though deciding SIFA-exploitability needs the ineffective-conditioned-bias test

is the permutation compute and the empirical floor $1/(n_{\mathrm{perm}} + 1)$, which for wide-null classifiers means n_{perm} must scale with site count. The design mitigates this with a Gaussian-tail refinement that is sharp exactly for the structural detector (whose leaks are most often isolated), and keeps `-legacy-thresholds` for reproducibility.

Hand-rolled vs. library math.
The bug-prone $\mathbb{Z}_q$ NTT was replaced by `sympy`; the small finite-field ranks were kept in numpy (fast, on the hot path) but are oracle-tested against `galois`. The binding constraint is free-threading: every runtime dependency must be pure-Python or GIL-safe, so `galois`/`numba` are test-only.

Processes vs. free threads.
Processes are simple and robust (no shared C state) but cost $\sim$11$\times$ the memory; free threads are memory-frugal but require CPython 3.14t and a GIL-safe dependency set. The platform keeps both on rebase-friendly branches so the choice is the operator's.

14 Limitations and Future Work

The audit identified four extension axes; the first has been built.

1. **Calibrated detectors** *(done, branch* `calibrated-detectors`*)* — §9.
2. **Fault-model sweep.** The detector sweep currently injects only `SKIP`; the six-model injector (`SET_MEM`, `BITFLIP_*`, `FLIP_FLAGS`) exists but is wired only to the golden-comparison campaign. Unifying them would let the detector repertoire run over data-corruption and branch-inversion faults—directly enabling MAYO vinegar-zeroization and seed-fixing, and Dilithium check-bypass, which the skip-only sweep cannot express.
3. **ML-KEM profile.** The KEM scheme layer and `ml-kem-768` firmware are present but no profile drives them; adding one proves genericity across the SIGN/KEM axis and exercises the detectors on Kyber decapsulation faults.
4. **Known-answer matrix + protected-vs-unprotected diff.** Pin each documented attack to an expected calibrated verdict (true-positive and golden true-negative), and add a mode that runs the same sweep on a hardened and an unprotected ELF, reporting which leaks a countermeasure closes, leaves, or newly introduces—the named use case.

Other known limits: `structural funcskip` is slow (∼3–4 min) due to the per-signature `sympy` ring deconvolution in featurisation (not the calibration); and for very large classifier sweeps with an isolated leak, n_{perm} should be raised until the floor clears q/m.

15 Conclusion

`ucpqc` is a compact (∼5.3 k LoC) but complete platform for systematic, emulation-based fault-leakage analysis of PQC firmware. Its layered design confines scheme knowledge to a thin profile layer, so the emulator, fault injector, capture-replay engine, detector repertoire, and now the statistical calibration are all reused unchanged across lattice and multivariate schemes. The recent calibration work turns every LEAK verdict into a permutation- or analytically-derived p-value with a sweep-wide false-discovery guarantee, eliminating four magic thresholds and any per-scheme tuning, while preserving every validated detection. The results are faithful in both directions: the platform pinpoints Dilithium's mask-removal leak at 100% (and its calibrated $p \approx 10^{-3}$ / structural $p = 0$), and it reports an honest negative on MAYO that names the exact missing capability—a data-fault sweep—as the next step.

A CLI Reference

```
python -m ucpqc info      fw.elf                 # scheme binding + memory map
python -m ucpqc roundtrip fw.elf                 # one keygen/sign/verify (or KEM) cycle
python -m ucpqc sweep     fw.elf --n 24          # whole-call leak sweep (ALAFA)
python -m ucpqc funcskip  fw.elf --target polyvecl_add   # intra-function skip sweep
python -m ucpqc fault     fw.elf --func challenge --model set_mem   # golden-comparison campaign
# calibration flags (sweep / funcskip):
#   --detector two_key|per_coord|subspace|structural|mmd|uniformity|spec_aware
#   --n-perm 1000  --fdr 0.01  --correction bh|holm  --legacy-thresholds
```

B Test and Verification Summary

- 32 system tests (`tests/run_tests.py`) — pass on CPython 3.12 and free-threaded 3.14t (GIL verified off).
- Field oracle (`tests/test_field_oracle.py`) — $\mathbb{Z}_q$ and GF(16) ranks matched against `galois` (3.12 only).
- Calibration self-tests — permutation calibration and generalisation, Benjamini–Hochberg / Holm control, and the `per_coord` p-value/threshold duality.
- Regression — calibrated verdicts reproduce the two `funcskip` leak sites and the ALAFA leak; `-legacy-thresholds` reproduces the pre-calibration results.